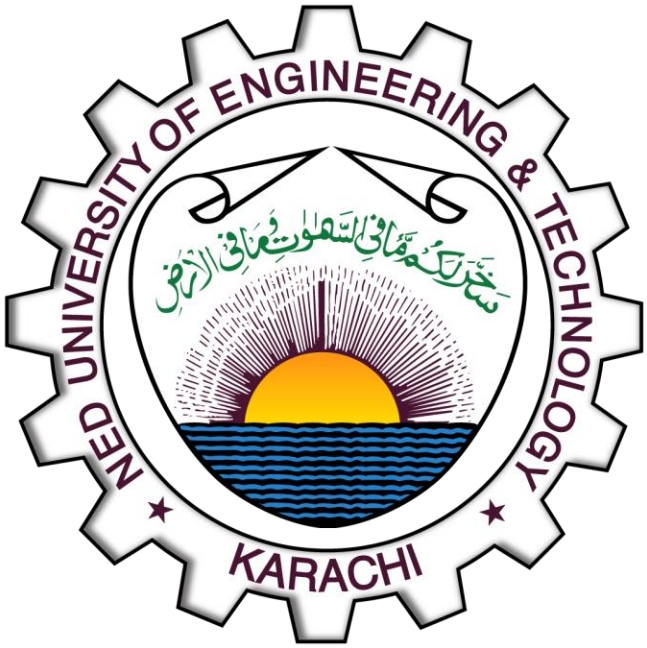
OBJECT ORIENTED PROGRAMMING

[CT-260]



NAME: TAHA AHMED MALLICK

ROLL NO: CT-25183

DEPARTMENT: BCIT **BATCH:** 2025

YEAR & SECTION: FSCS-D

LAB : #08

QUESTION#1:

CODE:

```
#include <iostream>
using namespace std;

class Vehicle
{
    string type, make, color, model;
    int year, miles;

public:
    Vehicle(string type, string make, string color, string model, int year, int
miles) : type(type), make(make), color(color), model(model), year(year), miles(miles)
{}

    void displayVehicle()
    {
        cout << "INFO:-" << endl
             << "Type: " << type << endl
             << "Make: " << make << endl
             << "Model: " << model << endl
             << "Color: " << color << endl
             << "Year: " << year << endl
             << "Miles: " << miles << endl;
    }
};

class GasVehicle : virtual public Vehicle
{
    float tankSize;

public:
    GasVehicle(string type, string make, string color, string model, int year, int
miles, float tankSize) : Vehicle(type, make, color, model, year, miles),
tankSize(tankSize) {}

    void displayGasVehicle()
    {
        cout << "Fuel Tank capacity: " << tankSize << "ltr" << endl;
    }
};

class ElectricVehicle : virtual public Vehicle
{
    float energyStorage;
```

```
public:
    ElectricVehicle(string type, string make, string color, string model, int year,
int miles, float energy) : Vehicle(type, make, color, model, year, miles),
energyStorage(energy) {}

    void displayElectricVehicle()
    {
        cout << "Energy storage: " << energyStorage << "kWh" << endl;
    }
};

class HighPerformace : public GasVehicle
{
    int hp, topSpeed;

public:
    HighPerformace(string type, string make, string color, string model, int year,
int miles, float tankSize, int hp, int topSpeed) : Vehicle(type, make, color, model,
year, miles), GasVehicle(type, make, color, model, year, miles, tankSize), hp(hp),
topSpeed(topSpeed) {}

    void displayHighPerformance()
    {
        displayVehicle();
        displayGasVehicle();
        cout << "Engine Power: " << hp << "hp" << endl
             << "Top speed: " << topSpeed << endl;
    }
};

class SportsCar : public HighPerformace
{
public:
    string gearBox, driveSys;

    SportsCar(string type, string make, string color, string model, int year, int
miles, float tankSize, int hp, int topSpeed, string gearBox, string driveSys) :
Vehicle(type, make, color, model, year, miles), HighPerformace(type, make, color,
model, year, miles, tankSize, hp, topSpeed), gearBox(gearBox), driveSys(driveSys) {}

    void dislplaySportsCar()
    {
        displayHighPerformance();
        cout << "Gear Box: " << gearBox << endl
             << "Drive System: " << driveSys << endl;
    }
};
```

```
class HeavyVehicle : public GasVehicle, public ElectricVehicle
{
    int maxWeight, wheels, length;

public:
    HeavyVehicle(string type, string make, string color, string model, int year, int
miles, float tankSize, float energy, int maxWeight, int wheels, int length) :
Vehicle(type, make, color, model, year, miles), GasVehicle(type, make, color, model,
year, miles, tankSize), ElectricVehicle(type, make, color, model, year, miles,
energy), maxWeight(maxWeight), wheels(wheels), length(length) {}

    void displayHeavyVehicle()
    {
        displayVehicle();
        displayGasVehicle();
        displayElectricVehicle();
        cout << "Max Weight: " << maxWeight << "kgs" << endl
             << "Number of wheels: " << wheels << endl
             << "Length: " << length << "m" << endl;
    }
};

class ConstructionTruck : public HeavyVehicle
{
public:
    string cargo;

    ConstructionTruck(string type, string make, string color, string model, int year,
int miles, float tankSize, float energy, int maxWeight, int wheels, int length,
string cargo) : Vehicle(type, make, color, model, year, miles), HeavyVehicle(type,
make, color, model, year, miles, tankSize, energy, maxWeight, wheels, length),
cargo(cargo) {}

    void displayConstructionTruck()
    {
        displayHeavyVehicle();
        cout << "Cargo: " << cargo << endl;
    }
};

class Bus : public HeavyVehicle
{
    int numSeats;

public:
    Bus(string type, string make, string color, string model, int year, int miles,
float tankSize, float energy, int maxWeight, int wheels, int length, int numSeats) :
```

```
Vehicle(type, make, color, model, year, miles), HeavyVehicle(type, make, color,
model, year, miles, tankSize, energy, maxWeight, wheels, length), numSeats(numSeats)
{}

    void displayBus()
    {
        displayHeavyVehicle();
        cout << "Number of seats: " << numSeats << endl;
    }
};

int main()
{
    Bus bus("Coach", "HINO", "White", "Hino Melpha", 2022, 25000, 500, 150, 500, 4,
60, 80);
    bus.displayBus();
    return 0;
}
```

OUTPUT:

```
INFO:-
Type: Coach
Make: HINO
Model: Hino Melpha
Color: White
Year: 2022
Miles: 25000
Fuel Tank capacity: 500ltr
Energy storage: 150kWh
Max Weight: 500kgs
Number of wheels: 4
Length: 60m
Number of seats: 80
```

QUESTION#2:

CODE:

```
#include <iostream>
#include <vector>
using namespace std;

class Character
{
protected:
    string name;
    int level, health;

public:
    Character(string name, int lvl, int hp) : name(name), level(lvl), health(hp) {}

    void displayCharacter()
    {
        cout << "Character Info:" << endl
             << "===============" << endl
             << "Name: " << name << endl
             << "Level: " << level << endl
             << "Health: " << health << endl;
    }

    virtual ~Character() {} // Best practice
};

class Warrior : virtual public Character
{
protected:
    int strength, meleeProf;

public:
    Warrior(string name, int lvl, int hp, int strength, int meleeProf)
        : Character(name, lvl, hp), strength(strength), meleeProf(meleeProf) {}

    void slash()
    {
        cout << "Warrior is slashing with power: "
             << strength + meleeProf * 0.2 << endl;
    }

    void displayWarrior()
    {
        cout << "Strength: " << strength << endl
```

```
            << "Melee Proficiency: " << meleeProf << endl;
    }

    void display()
    {
        displayCharacter();
        displayWarrior();
    }
};

class Mage : virtual public Character
{
protected:
    int intelligence, spellCastProf;

public:
    Mage(string name, int lvl, int hp, int intel, int prof)
        : Character(name, lvl, hp), intelligence(intel), spellCastProf(prof) {}

    void fireball()
    {
        cout << "Throwing fireball with power: "
             << spellCastProf * intelligence * 0.4 << endl;
    }

    void displayMage()
    {
        cout << "Intelligence: " << intelligence << endl
             << "Spell Cast Proficiency: " << spellCastProf << endl;
    }

    void display()
    {
        displayCharacter();
        displayMage();
    }
};

class Archer : virtual public Character
{
protected:
    int dexterity, rangedWepProf;

public:
    Archer(string name, int lvl, int hp, int dex, int prof)
        : Character(name, lvl, hp), dexterity(dex), rangedWepProf(prof) {}

    void rapidShot()
```

```
    {
        cout << "Shooting rapid arrows with accuracy: "
             << dexterity + rangedWepProf * 0.1
             << " and damage: "
             << rangedWepProf * 0.4 + dexterity * 0.1 << endl;
    }

    void displayArcher()
    {
        displayCharacter();
        cout << "Dexterity: " << dexterity << endl
             << "Ranged Weapon Proficiency: " << rangedWepProf << endl;
    }
};

class NPC : public Character
{
protected:
    vector<string> dialogues;
    vector<int> movements;

public:
    NPC(string name, int lvl, int hp, vector<string> dia, vector<int> moves)
        : Character(name, lvl, hp), dialogues(dia), movements(moves) {}

    void say(int i)
    {
        cout << "Saying: " << dialogues[i] << endl;
    }

    void move()
    {
        cout << "Moving..." << endl;
        for (int i = 0; i < movements.size(); i++)
            cout << movements[i] << endl;
    }

    void displayNPC()
    {
        displayCharacter();
        cout << "Dialogues:" << endl;
        for (int i = 0; i < dialogues.size(); i++)
            say(i);

        cout << "Moves:" << endl;
        move();
    }
};
```

```
class Mighty : public Warrior, public Mage
{
public:
    Mighty(string name, int lvl, int hp, int strength, int meleeProf, int intel, int
prof)
        : Character(name, lvl, hp),
          Warrior(name, lvl, hp, strength, meleeProf),
          Mage(name, lvl, hp, intel, prof) {}

    void spellSlash()
    {
        cout << "Using abilities of both Warrior and Mage." << endl;
    }

    void displayMighty()
    {
        displayCharacter();
        displayWarrior();
        displayMage();
    }
};

int main()
{
    Warrior warrior("Warrior", 5, 100, 120, 50);
    Mage mage("Mage", 2, 70, 57, 68);
    Archer archer("Archer", 10, 23, 54, 90);

    vector<string> dialogues = {"Hi! I am a character","Wanna go for an adventure?",
"Bye!!"};

    vector<int> moves = {1, 2, 1, 3, 4, 2};

    NPC npc("NPC", 0, 89, dialogues, moves);

    Mighty mighty("Mighty", 90, 150, 55, 130, 200, 60);

    warrior.display();
    cout << "Abilities:" << endl;
    warrior.slash();

    cout << endl;

    mage.display();
    cout << "Abilities:" << endl;
    mage.fireball();
```

```
    cout << endl;

    archer.displayArcher();
    cout << "Abilities:" << endl;
    archer.rapidShot();

    cout << endl;

    npc.displayNPC();

    cout << endl;

    mighty.displayMighty();
    cout << "Abilities:" << endl;
    mighty.spellSlash();

    return 0;
}
```

OUTPUT:

```
Character Info:
===============
Name: Warrior
Level: 5
Health: 100
Strength: 120
Melee Proficiency: 50
Abilities:
Warrior is slashing with power: 130

Character Info:
===============
Name: Mage
Level: 2
Health: 70
Intelligence: 57
Spell Cast Proficiency: 68
Abilities:
Throwing fireball with power: 1550.4

Character Info:
===============
Name: Archer
Level: 10
Health: 23
Dexterity: 54
Ranged Weapon Proficiency: 90
```

```
Abilities:
Shooting rapid arrows with accuracy: 63 and damage: 41.4

Character Info:
===============
Name: NPC
Level: 0
Health: 89
Dialogues:
Saying: Hi! I am a character
Saying: Wanna go for an adventure?
Saying: Bye!!
Moves:
Moving...
1
2
1
3
4
2

Character Info:
===============
Name: Mighty
Level: 90
Health: 150
Strength: 55
Melee Proficiency: 130
Intelligence: 200
Spell Cast Proficiency: 60
Abilities:
Using abilities of both Warrior and Mage.
```

QUESTION#3:

CODE:

```
#include <iostream>
using namespace std;

// Base Class
class Account
{
protected:
    double balance;

public:
    Account()
    {
        cout << "initial balance: ";
        cin >> balance;
    }

    Account(double balance)
    {
        this->balance = balance;
    }

    virtual void deposit(double amount)
    {
        balance += amount;
        cout << "Deposited: " << amount << endl;
    }

    virtual void withdraw(double amount)
    {
        if (amount <= balance)
        {
            balance -= amount;
            cout << "Withdrawn: " << amount << endl;
        }
        else
        {
            cout << "Insufficient balance!\n";
        }
    }

    void checkBalance()
    {
        cout << "Balance: " << balance << endl;
```

```
    }

    double getBalance()
    {
        return balance;
    }

    void setBalance(double b)
    {
        balance = b;
    }
};

// Derived Class: InterestAccount
class InterestAccount : public Account
{
protected:
    double interest;

public:
    InterestAccount() : Account() {}

    InterestAccount(double b) : Account(b) {}

    void deposit(double amount)
    {
        interest = amount * 0.30;
        balance += amount + interest;
        cout << "Deposited with interest (30%): " << amount + interest << endl;
    }
};

// Derived Class: ChargingAccount
class ChargingAccount : public Account
{
protected:
    double fee = 25;

public:
    ChargingAccount() : Account() {}

    ChargingAccount(double b) : Account(b) {}

    void withdraw(double amount)
    {
        double total = amount + fee;

        if (total <= balance)
```

```
        {
            balance -= total;
            cout << "Withdrawn: " << amount << " with fee Rs.25\n";
        }
        else
        {
            cout << "Insufficient balance including fee!\n";
        }
    }
};

class ACI
{
public:
    void transfer(double amount, Account &acc)
    {
        acc.deposit(amount);
        cout << "Transferred " << amount << " to Account\n";
    }

    void transfer(double amount, InterestAccount &acc)
    {
        acc.deposit(amount);
        cout << "Transferred " << amount << " to InterestAccount\n";
    }

    void transfer(double amount, ChargingAccount &acc)
    {
        acc.deposit(amount);
        cout << "Transferred " << amount << " to ChargingAccount\n";
    }
};

int main()
{
    cout << "Enter Account 1 ";
    Account acc1;
    cout << "Enter Account 2 ";
    InterestAccount acc2;
    cout << "Enter Account 3 ";
    ChargingAccount acc3;

    cout << endl;
    acc1.deposit(1000);
    acc2.deposit(1000);
    cout << endl;
    acc1.withdraw(500);
    acc3.withdraw(200);
```

```
    cout<<endl;

    cout << "Account 1:- " << endl;
    acc1.checkBalance();
    cout << "Account 2:- " << endl;
    acc2.checkBalance();
    cout << "Account 3:- " << endl;
    acc3.checkBalance();

    ACI obj;
    cout<<endl;
    obj.transfer(300, acc1);
    obj.transfer(300, acc2);
    obj.transfer(300, acc3);
    cout<<endl;

    cout << "Account 1:- " << endl;
    acc1.checkBalance();
    cout << "Account 2:- " << endl;
    acc2.checkBalance();
    cout << "Account 3:- " << endl;
    acc3.checkBalance();

    return 0;
}
```

OUTPUT:

```
Enter Account 1 initial balance: 500000
Enter Account 2 initial balance: 250000
Enter Account 3 initial balance: 125000

Deposited: 1000
Deposited with interest (30%): 1300

Withdrawn: 500
Withdrawn: 200 with fee Rs.25

Account 1:-
Balance: 500500
Account 2:-
Balance: 251300
Account 3:-
Balance: 124775

Deposited: 300
Transferred 300 to Account
Deposited with interest (30%): 390
```

```
Transferred 300 to InterestAccount
Deposited: 300
Transferred 300 to ChargingAccount

Account 1:-
Balance: 500800
Account 2:-
Balance: 251690
Account 3:-
Balance: 125075
```

QUESTION#4:

CODE:

```
#include <iostream>
using namespace std;

class Media {
public:
    virtual void borrow() = 0;
    virtual void returning() = 0;
    virtual void display() = 0;
};

class BookAttr {
protected:
    string name, author;
    int numPage;
public:
    BookAttr(string name, string auth, int pg) : name(name), author(auth),
numPage(pg) {}
};

class MagzineAttr {
protected:
    string editor;
    int revision, issueNum;
public:
    MagzineAttr(string editor, int num, int rev) : editor(editor), issueNum(num),
revision(rev) {}
};

class DVDAttr {
protected:
    string name, director;
    int numAlbums;
public:
    DVDAttr(string name, string director, int numAlbums) : name(name),
director(director), numAlbums(numAlbums) {}
};

class Book : protected Media, protected BookAttr {
public:
    Book(string name, string auth, int pg) : BookAttr(name, auth, pg) {}
    void borrow() {
        cout << "Borrowing " << name << " by: " << author << endl;
    }
```

```
    void returning() {
        cout << "Returning " << name << endl;
    }
    void display() {
        cout << "INFO:-" << endl;
        cout << "Name: " << name << endl;
        cout << "Author: " << author << endl;
        cout << "Pages: " << numPage << endl;
    }
};

class Magzine : protected Media, protected MagzineAttr {
public:
    Magzine(string editor, int num, int rev) : MagzineAttr(editor, num, rev) {}
    void borrow() {
        cout << "Borrowing Magzine number " << issueNum << " by: " << editor << endl;
    }
    void returning() {
        cout << "Returning Magzine " << issueNum << " Revision: " << revision <<
endl;
    }
    void display() {
        cout << "INFO:-" << endl;
        cout << "Issue: " << issueNum << endl;
        cout << "Revision: " << revision << endl;
        cout << "Editor: " << editor << endl;
    }
};

class DVD : protected Media, protected DVDAttr {
public:
    DVD(string name, string director, int numAlbums) : DVDAttr(name, director,
numAlbums) {}
    void borrow() {
        cout << "Borrowing " << name << " by: " << director << endl;
    }
    void returning() {
        cout << "Returing " << name << endl;
    }
    void display() {
        cout << "INFO:-" << endl;
        cout << "Name: " << name << endl;
        cout << "Director: " << director << endl;
        cout << "Albums: " << numAlbums << endl;
    }
};

int main() {
```

```
    Book b1("ABC", "CDE", 56);
    Magzine m1("Student Digest", 123, 456);
    DVD d1("XYZ", "PQR", 5);
    b1.display();
    m1.display();
    d1.display();
    cout << endl;
    b1.borrow();
    return 0;
}
```

OUTPUT:

```
INFO:-
Name: ABC
Author: CDE
Pages: 56
INFO:-
Issue: 123
Revision: 456
Editor: Student Digest
INFO:-
Name: XYZ
Director: PQR
Albums: 5

Borrowing ABC by: CDE
```